MOBILUX

Fix. Track. Trust.

A Smart Phone Repair Management & E-Commerce Platform

SOFTWARE PROJECT PROPOSAL

Prepared by

Gayash

mastergayash43@gmail.com

Colombo, Sri Lanka | July 2026

Table of Contents

1. Executive Summary 3
2. Problem Statement 3
3. Project Objectives 3
4. Project Scope 4
5. System Actors 4
6. Functional Requirements 5
7. Non-Functional Requirements 5
8. Proposed System Architecture 6
9. Database Design 6
10. Use Case Diagram 7
11. UI/UX Design Approach 8
12. Recommended Technology Stack 9
13. Payment Integration 10
14. Loyalty & Discount System 10
15. Future Roadmap — Multi-Tenant SaaS Vision 10
16. Project Timeline 10
17. Conclusion 11
18. Appendix — Glossary 11

1. Executive Summary

Mobilux is a web-based platform designed to digitize the day-to-day operations of a phone repair business. It replaces manual, paper- and chat-based repair tracking with a structured system where customers submit repair requests online, follow live status updates, pay securely in Sri Lankan Rupees (LKR), and earn loyalty discounts for repeat visits. The shop owner manages every ticket, payment, and product from a single admin dashboard, and can additionally sell accessories — and, when ready, pre-owned phones — through an integrated storefront.

The project is currently scoped for a single, home-based repair operation run by one owner. However, the system is deliberately architected around role-based access control from day one, so that a third actor — shop staff such as a cashier or technician — can be introduced later with no structural rework. This positions Mobilux not just as an internal tool, but as the foundation of a licensable, multi-tenant SaaS product that can eventually be sold to other independent repair shops.

2. Problem Statement

Small, independent phone repair businesses in Sri Lanka — including home-based operations — typically rely on manual, informal processes that limit their professionalism, scalability, and customer trust:

- Repair status is tracked through phone calls, WhatsApp messages, or handwritten logs, creating confusion and repeated customer follow-ups.
- Payments are cash-only or via ad-hoc bank transfers with no digital receipt trail or automatic confirmation.
- There is no structured way to reward loyal, repeat customers, so retention relies purely on personal relationships.
- Accessories and spare devices are sold informally, with no online catalog or storefront to reach customers outside the shop's immediate circle.
- The business has no professional web presence, which limits credibility when competing against established repair chains.

Mobilux addresses each of these gaps with a single, purpose-built platform — while being engineered to scale beyond one owner and one shop.

3. Project Objectives

1. Provide customers a self-service portal to submit repair requests, track status in real time, and pay online in LKR.
2. Replace informal, chat-based tracking with a structured, auditable digital ticketing system.
3. Enable secure online payments via a local payment gateway, with a manual bank-transfer and receipt-upload option as a fallback.
4. Automatically reward repeat customers through a configurable, rule-based loyalty discount engine.
5. Give the shop owner a single dashboard to manage repair tickets, customers, inventory, payments, and reports.
6. Introduce an accessories and phone storefront that the owner can show or hide independently of the repair service.
7. Architect the platform with role-based access control from day one so it can scale into a licensable, multi-tenant SaaS product for other repair shops.

8. Deliver a clean, modern, mobile-responsive interface that builds customer trust and reflects a professional brand.

4. Project Scope

4.1 Phase 1 — In Scope (Current Deployment)

The initial release targets a single, home-based repair shop operated by one owner, and includes:

- Public marketing home page introducing the shop, services, and accessories/phones on sale.
- Customer registration, login, and profile management.
- Repair request submission with device details, issue description, and optional photos.
- Real-time repair status tracking with a defined status pipeline.
- Online payment via a local LKR payment gateway, plus bank-transfer receipt upload with manual verification.
- Customer dashboard showing repair history, active tickets, invoices, and loyalty discount progress.
- Accessories and phone catalog with an admin-controlled visibility toggle for phone sales.
- Admin dashboard for managing tickets, customers, products, discount rules, payments, and basic reports.

4.2 Future Phases — Out of Scope for Phase 1

The following are intentionally deferred but planned for in the system's architecture (see Section 12, Future Roadmap):

- A dedicated Staff role (Cashier / Technician) with permission-scoped access.
- Multi-tenant conversion so the platform can be licensed to other repair shops as a subscription product.
- Native mobile applications (the web app will be fully responsive in the meantime).
- Two-way WhatsApp/SMS chat integration and courier/delivery tracking.

5. System Actors

Mobilux is designed around three actors. Two are active from launch; the third is a forward-looking role already reflected in the data model and permission system so it can be switched on without re-architecting the platform.

Actor	Status	Responsibilities
Admin / Owner	Active — Phase 1	Full control: manage repair tickets, customers, products, discount rules, payments, reports, and (later) staff accounts.
Customer	Active — Phase 1	Register/login, submit repair requests, track status, pay online or upload a bank receipt, view dashboard and repair history, redeem loyalty discounts, browse and buy accessories/phones.
Staff — Cashier / Technician	Planned — Phase 2	Technician: update repair progress and parts used. Cashier: process in-store payments and verify uploaded receipts. Both operate under owner-defined permissions.

6. Functional Requirements

6.1 Customer-Facing Requirements

ID	Requirement
FR-01	Customer can register, log in, and manage a personal profile.
FR-02	Customer can submit a repair request with device type, brand, model, issue description, and optional photos.
FR-03	Customer can track live repair status (Received → Diagnosing → Awaiting Parts → In Progress → Completed → Ready for Pickup).
FR-04	Customer can pay online in LKR through the integrated payment gateway.
FR-05	Customer can alternatively pay via bank transfer and upload a receipt for manual verification.
FR-06	Customer can view a dashboard summarizing active and past repairs, invoices, and loyalty status.
FR-07	Customer automatically receives a discount once configured repair-count thresholds are met.
FR-08	Customer can browse and purchase accessories, and phones when the owner makes that section visible.
FR-09	Customer receives email notifications when a repair's status changes.

6.2 Admin-Facing Requirements

ID	Requirement
FR-10	Admin can view, create, and update repair tickets, and assign a status at each stage.
FR-11	Admin can manage the product catalog (accessories and phones) including price, stock, and images.
FR-12	Admin can independently toggle visibility of the "Phones for Sale" storefront section.
FR-13	Admin can configure loyalty discount rules (e.g., repair-count thresholds and discount percentages).
FR-14	Admin can verify or reject bank-transfer receipts submitted by customers.
FR-15	Admin can view sales and repair reports (revenue, repair volume, common issues).
FR-16	Admin can create and manage staff accounts with scoped permissions (Phase 2).

7. Non-Functional Requirements

Category	Requirement
Security	Password hashing (bcrypt/argon2), HTTPS everywhere, JWT-based sessions, role-based access control, PCI-compliant payment handling via the payment gateway.
Performance	Key pages load in under 2 seconds on a typical mobile connection in Sri Lanka.
Responsiveness	Fully responsive, mobile-first layout — the majority of customers are expected to browse on mobile.
Localization	All prices displayed in LKR; dates/times in Asia/Colombo timezone.

Category	Requirement
Scalability	Database schema designed with future multi-tenant partitioning in mind (tenant-ready keys).
Usability	Clean, minimal interface requiring no training for a first-time customer to submit a repair request.
Maintainability	Type-safe codebase (TypeScript end-to-end) with a modular, documented architecture.
Availability	Target 99.5% uptime using managed cloud hosting with automatic recovery.

8. Proposed System Architecture

Mobilux follows a modern three-layer architecture: a client layer used by customers and the owner, an application layer built on Next.js that serves both the UI and the API, and a data & integration layer that persists information and talks to external services such as the payment gateway and notification providers. This separation keeps the codebase simple enough for a solo developer to ship quickly today, while remaining structured enough to split into independent services if the platform grows into a multi-tenant SaaS product.

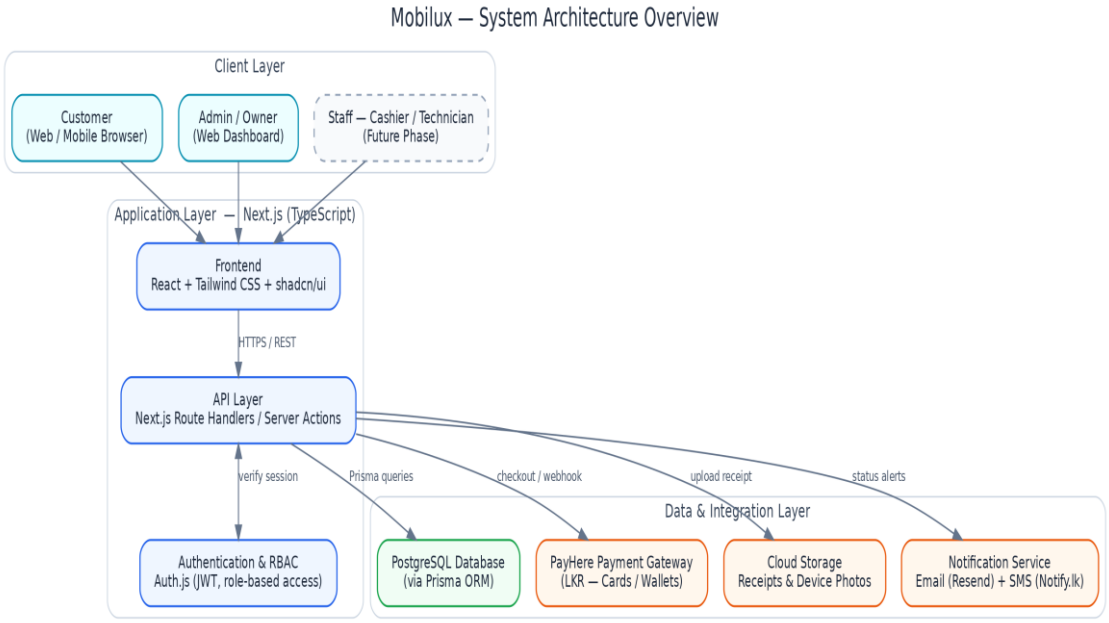


Figure 1 — High-level system architecture.

All client requests pass through a single authenticated API layer, which enforces role-based permissions before reaching the database, the payment gateway, cloud storage, or the notification service — keeping business rules centralized in one place rather than duplicated across the frontend.

9. Database Design

The core schema centers on the User, RepairTicket, and Payment entities, with Product, Order, and OrderItem supporting the accessories/phone storefront, and DiscountRule and Notification supporting the loyalty and alerting features. Every entity uses a surrogate primary key and standard foreign-key relationships, keeping the schema simple to extend with a tenant identifier in Phase 3.

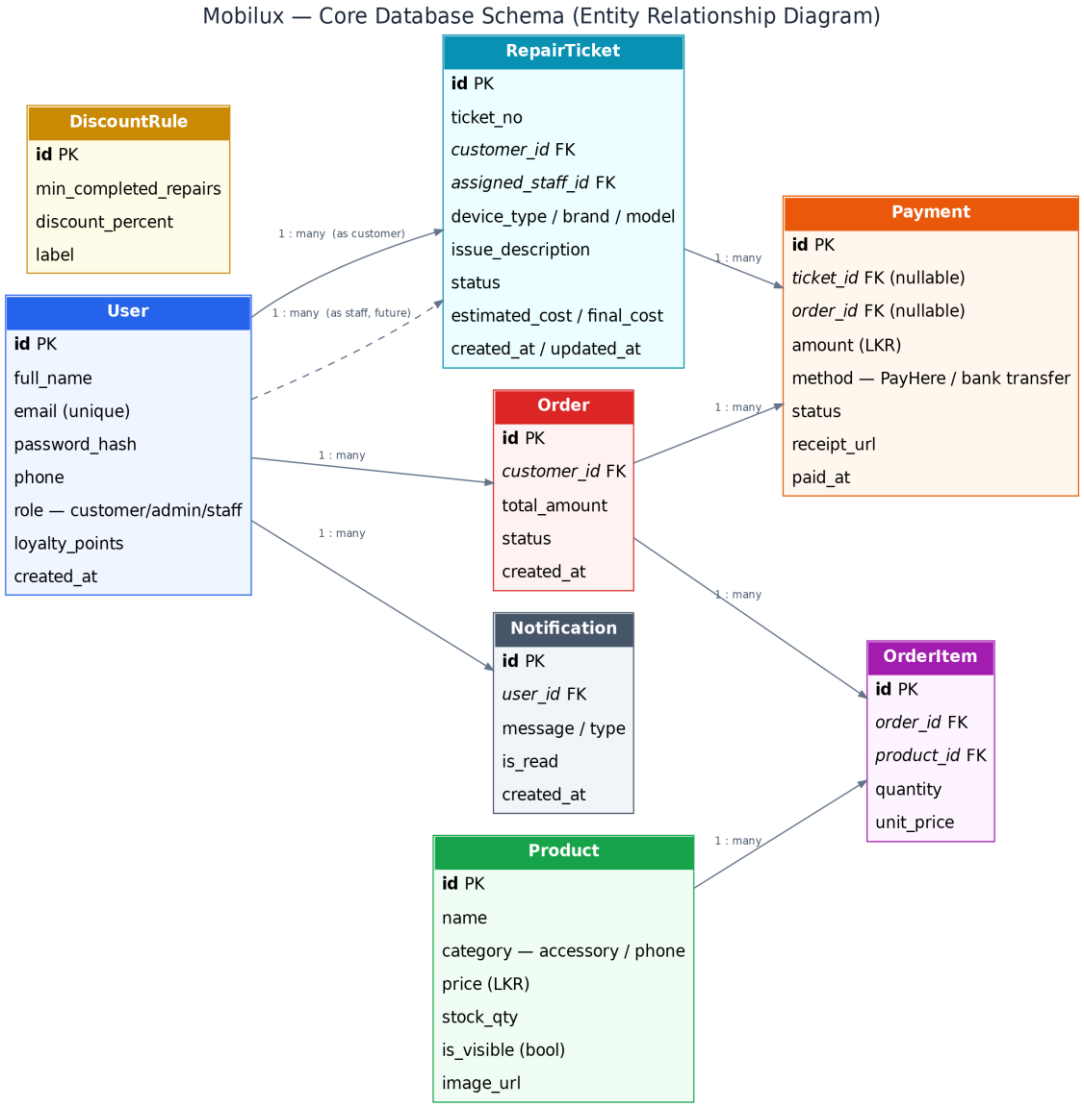


Figure 2 — Entity-relationship diagram of the core schema.

10. Use Case Diagram

The diagram below summarizes how each actor interacts with the system. Customer and Admin use cases are active in Phase 1; the Staff actor and its use cases (dashed) are planned for Phase 2 and are already reflected in the permission model.

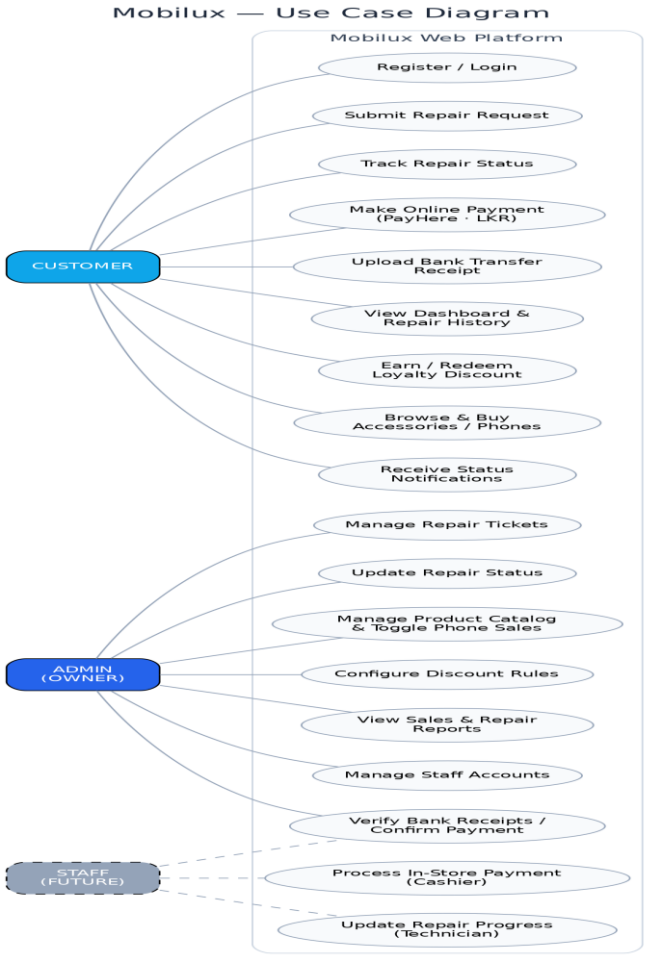


Figure 3 — Use case diagram for Customer, Admin, and the future Staff role.

11. UI/UX Design Approach

Mobilux's interface is designed to feel modern and trustworthy without being visually loud — appropriate for a service business where customers are often anxious about a broken device and want clarity, not clutter.

11.1 Design Principles

- Clean and minimal: generous white space, card-based layouts, and a restrained color palette keep focus on repair status and key actions.
- Consistent component system: built with a shared design system (shadcn/ui) so buttons, forms, and status badges look and behave identically across every page.
- Mobile-first: layouts are designed for small screens first, then progressively enhanced for tablet and desktop.
- Clear status communication: color-coded badges and progress indicators make repair status understandable at a glance.
- Accessible by default: color contrast, focus states, and font sizes follow WCAG AA guidelines.

11.2 Visual Language

Token	Value	Usage
Primary	#2563EB (Blue)	Primary actions, links, active navigation

Token	Value	Usage
Accent	#0EA5E9 (Sky)	Highlights, badges, secondary emphasis
Success	#16A34A (Green)	Completed repairs, successful payments
Warning	#EA580C (Orange)	Pending verification, awaiting parts
Text / Dark	#1E293B (Slate)	Body text and headings
Surface	#F8FAFC (Slate 50)	Page and card backgrounds
Typography	Inter / Poppins	Inter for body text, Poppins for headings

Prototypes will be designed in Figma before implementation, covering the home page, repair tracker, customer dashboard, checkout flow, and admin panel across mobile and desktop breakpoints.

12. Recommended Technology Stack

The stack below was chosen using three criteria: it is currently in high industry demand (strengthening the CV value of this project), it lets a solo developer ship a production-quality product quickly, and it scales cleanly toward the multi-tenant SaaS vision without a rewrite.

Layer	Technology	Why
Frontend	Next.js (App Router) + React + TypeScript	Industry-standard React framework; server-side rendering improves SEO for the public home page; TypeScript reduces bugs across a solo codebase.
Styling / UI	Tailwind CSS + shadcn/ui	Utility-first styling and a proven, accessible component library enable a clean, minimal UI without a dedicated designer.
Backend / API	Next.js Route Handlers & Server Actions (Node.js)	One codebase for frontend and backend — faster to build and deploy as a solo developer, still fully split-able into microservices later.
Database	PostgreSQL	Reliable relational integrity for tickets, payments, and orders; scales to a multi-tenant schema in Phase 3.
ORM	Prisma	Type-safe database access and simple, version-controlled migrations.
Authentication	Auth.js (NextAuth)	Secure JWT-based sessions with role-based access control built in — ready for the Staff role.
Payments	PayHere	Sri Lanka’s Central-Bank-approved, PCI DSS Level 1 certified payment gateway with native LKR support and local cards/wallets (Visa, Mastercard, eZ Cash, FriMi, Genie, Vishwa).
File Storage	Cloudinary / AWS S3	Stores bank-receipt uploads and device photos reliably outside the application server.
Notifications	Resend (email) + Notify.lk (SMS)	Automated status alerts to customers via email and, optionally, local SMS.
Hosting	Vercel + Neon (managed Postgres)	Zero-ops deployment with a generous free tier, ideal for a bootstrapped MVP.
Design & CI	Figma, GitHub, GitHub Actions	Prototyping before build, version control, and automated build/test/deploy pipelines.

13. Payment Integration

Two payment paths are supported so that customers can pay however they are most comfortable, while the owner retains full visibility and control:

13.1 Online Payment (PayHere)

The customer pays instantly through PayHere's hosted checkout using a card, mobile wallet, or online banking, in LKR. PayHere confirms the transaction via webhook, and the ticket or order is marked "Paid" automatically — no manual reconciliation required. Transaction fees are approximately 2.7%–3.3% depending on plan tier, with no setup cost.

13.2 Bank Transfer + Receipt Upload

For customers who prefer a direct bank transfer, the customer uploads a photo or PDF of the transfer receipt against their ticket or order. The payment is flagged "Pending Verification" until the admin reviews and confirms it from the dashboard, after which the customer is notified and the ticket proceeds.

14. Loyalty & Discount System

Mobilux includes a configurable, rule-based loyalty engine rather than a hard-coded discount. The owner defines thresholds — for example, a 5% discount from the 3rd completed repair, and 10% from the 5th — through the admin panel's DiscountRule settings. The system automatically tracks each customer's completed-repair count and applies the correct discount at checkout, with the customer's dashboard showing progress toward their next reward. This keeps the incentive structure flexible as the business grows or runs promotions.

15. Future Roadmap — Multi-Tenant SaaS Vision

Mobilux is scoped today as a single-shop tool, but every architectural decision — role-based access control, a Staff actor already modeled in the permission system, and a schema designed for tenant partitioning — is made with a longer-term goal in mind: turning Mobilux into software that can be licensed to other independent repair shops.

Phase	Milestone	Description
Phase 1	Single-Shop MVP (this proposal)	Owner and Customer actors; core repair tracking, payments, loyalty, and storefront features for one home-based shop.
Phase 2	Staff & Multi-User Access	Introduce Cashier and Technician roles with permission-scoped dashboards and shift-based ticket assignment.
Phase 3	Multi-Tenant SaaS	Partition the schema by shop (tenant), add subscription billing, white-label branding, and a shop-onboarding flow — enabling Mobilux to be sold as a product to other repair businesses.

16. Project Timeline

The following is an estimated 12-week delivery plan for Phase 1, assuming solo, part-time development:

Milestone	Duration	Key Deliverables
1. Discovery & Design	Weeks 1–2	Finalized requirements, Figma UI mockups, database schema design.
2. Core Platform	Weeks 3–6	Authentication, home page, repair ticketing, customer & admin dashboards.
3. Payments & Loyalty	Weeks 7–8	PayHere integration, bank-receipt upload flow, discount engine.
4. Storefront & Notifications	Weeks 9–10	Accessories/phone catalog with visibility toggle, email/SMS alerts.
5. Testing & Launch	Weeks 11–12	QA, user acceptance testing, deployment, go-live.

17. Conclusion

Mobilux replaces the manual, cash-based operations of a home-based phone repair business with a modern, trustworthy digital platform — giving customers visibility and convenience, and giving the owner a single dashboard to run the business. Just as importantly, the project is designed beyond its immediate use case: its role-based architecture, tenant-ready data model, and industry-standard technology stack lay the groundwork for Mobilux to grow from an internal tool into a licensable SaaS product for repair shops across Sri Lanka and beyond.

18. Appendix — Glossary

Term	Meaning
RBAC	Role-Based Access Control — restricting system actions based on a user's assigned role.
ORM	Object-Relational Mapping — a tool (Prisma) that lets code interact with the database using type-safe objects instead of raw SQL.
JWT	JSON Web Token — a compact, signed token used to securely identify a logged-in user.
LKR	Sri Lankan Rupee — the currency used for all pricing and transactions in Mobilux.
PayHere	A Central Bank of Sri Lanka-approved online payment gateway supporting LKR transactions.
Tenant	In a multi-tenant SaaS, a single customer organization (e.g., one repair shop) whose data is isolated from others sharing the same platform.
MVP	Minimum Viable Product — the smallest complete version of the product worth releasing.